

Evaluating Retrieval-Augmented Generation Frameworks Under Local Deployment Constraints

A Comparative Study of Single-Retrieval and Corrective Retrieval Pipelines

Fadi Mardelli · **George-Adelin Aniței**

Promotor: Prof. Dr. Mihai Bâce

Motivation: Local Deployment with Open-Weight Models

Cloud deployment

- + larger models
- + managed infrastructure
- + easy scaling and maintenance
- data leaves local control
- network/provider dependence
- recurring API and subscription

Local deployment

- + data stays local
- + offline operation
- + control over corpus and model stack
- limited memory
- higher latency
- energy cost

Sources: Park et al. (2025),
Wang et al. (2025).

Local deployment

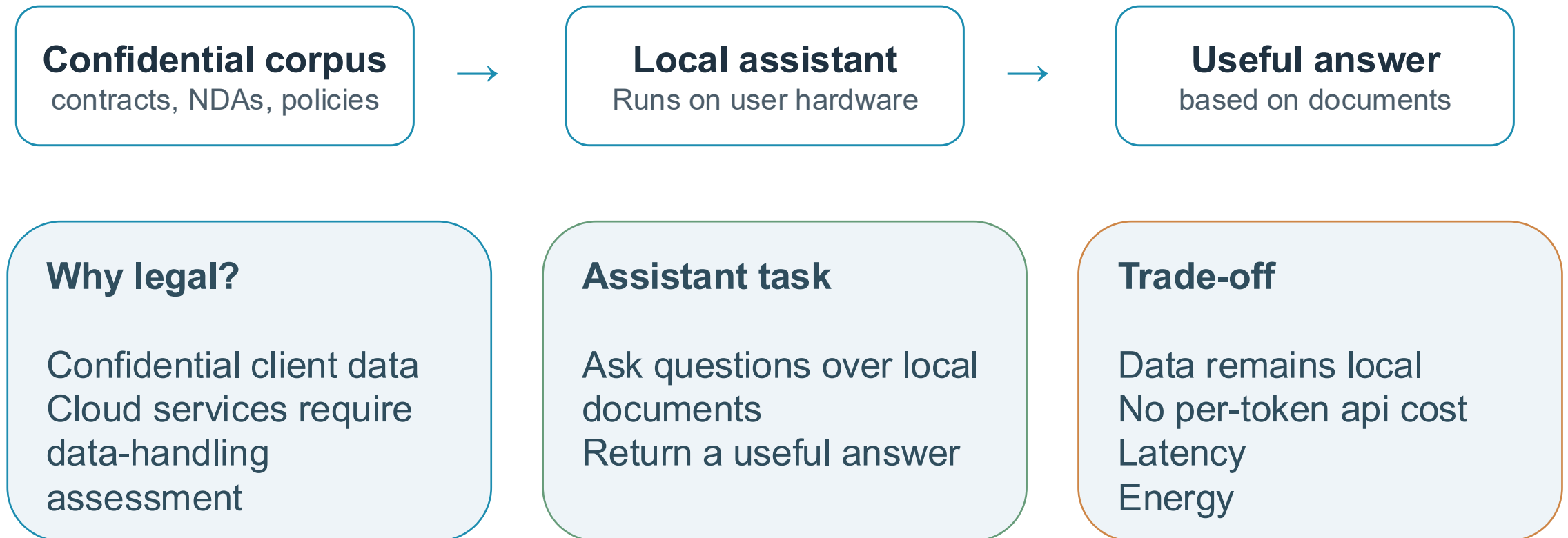
Different setups:

- Company servers
- Personal laptops

In this thesis :

- Laptop-class consumer hardware: MacBook Air M4.
- Offline, no remote API calls.
- Open weight models.

Use Case: Local Document Assistant

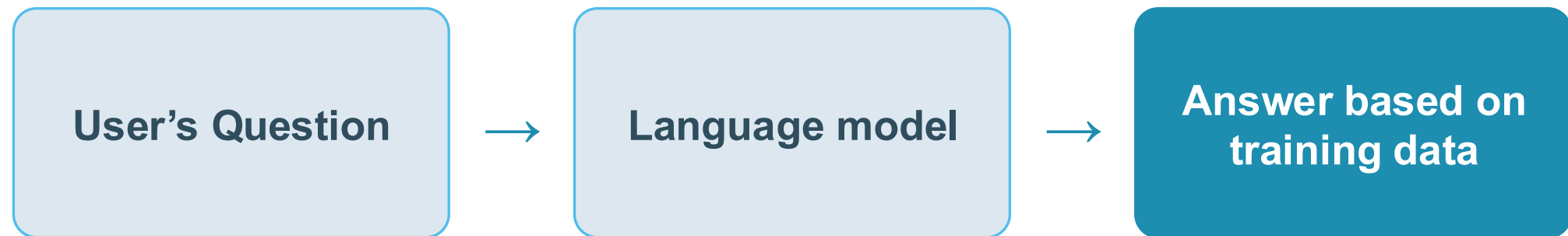


Sources: confidentiality and cloud-risk assessment (ABA, 2024; CCBE, 2025).

Language Models

- Do not retrieve facts
- Do not know today's information
- Do not know your specific document

→ **They may hallucinate - producing confident but incorrect answers**



The Solution: RAG

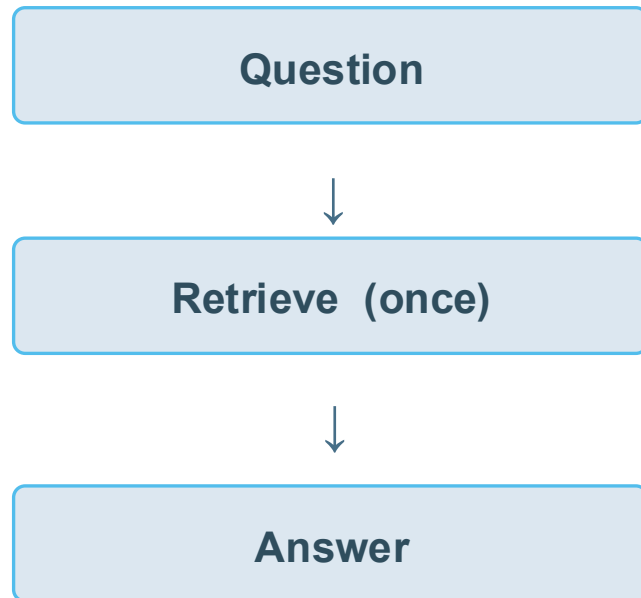
- Retrieval-Augmented Generation in 2020
- Multiple strategies, but one main high-level idea



Lewis et al. (2020).

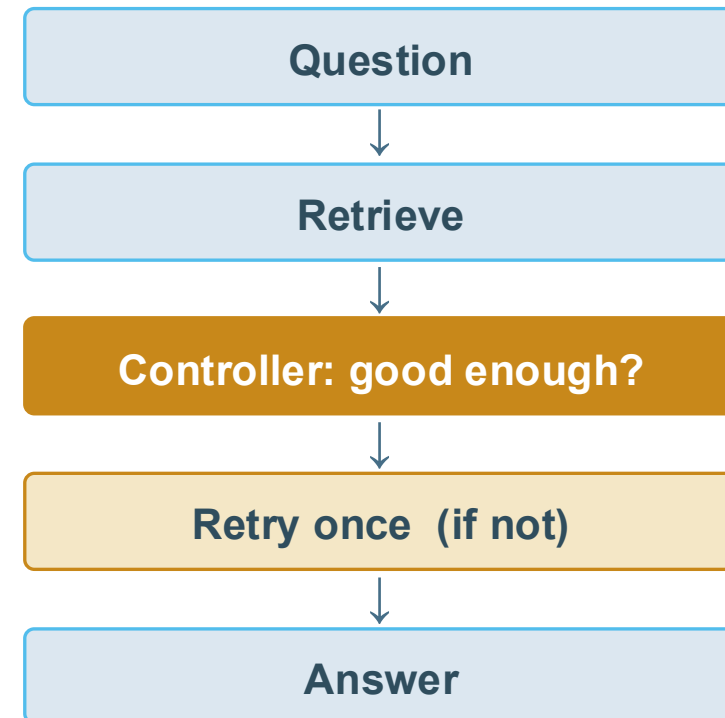
One-pass RAG and its blind spot

One-pass RAG



If the first retrieval misses, the answer is built on the wrong evidence with no way to recover.

Bounded corrective RAG



At most one corrective retry a bounded extra cost.

Research questions

RQ1 When does bounded corrective RAG improve over one-pass RAG under local hardware constraints and at what cost?

RQ2 How does task structure affect the value of bounded correction in legal evidence recovery and multi-document question answering ?

RQ3 How do model family, model scale, and deployment setting affect corrective performance and cost?

THE GAP

Corrective RAG is almost always tested with cloud models, server-class GPUs, and remote API .

Does a single bounded corrective pass earn its extra latency and compute when everything runs locally - on a laptop?



Our contributions.

Controlled Pipeline
Comparison

One-pass RAG vs.
Bounded Corrective
RAG

Cross-Domain
Benchmarking

LegalBench-mini vs
HotpotQA

Model Family &
Scale Analysis

Llama 3B/8B vs
Qwen 3B/7B

Deployment-oriented
perspective

Measure latency,
retrieval and answer
quality, memory.

Alternative RAG architectures

1. Self-Reflective RAG (Self-RAG) (Asai et al., 2024)
 - **retrieves, generates and critiques** its own outputs using reflection tokens and special prompts
 - **complex** and **resource-intensive**, impractical on a MacBook Air
2. Adaptive-RAG (Jeong et al., 2024)
 - vary the **number of retrieved passages** based on question complexity
 - **retrieval depth** may increase unpredictably, making **latency unpredictable** on local hardware.
3. FLARE (Active Retrieval) (Jiang et al., 2023)
 - active mid-generation retrieval based on low token confidence.
 - **Continuous** probability monitoring creates **unacceptable latency**.
4. IRCOT (Multi-step / Chain of Thought) (Trivedi et al., 2023)
 - Interleaves multi-step reasoning with repeated retrieval calls.
 - **Unbounded, multi-cycle generation loops** cause severe **thermal accumulation** and **massive latency spikes** on local hardware.

What is CRAG?

1. Uses a **fine-tuned retrieval evaluator** that chooses among three actions:
 - **Correct:** accept the passages and proceed.
 - **Ambiguous:** refine retrieval via query decomposition and re-ranking.
 - **Incorrect:** perform web search to bring in fresh documents.
2. Multiple iterations: allows complex correction loops.
3. Heavy dependencies: extra evaluator model, needs access to the web.

Sources: Yan et al., 2024

Positioning of our work

Original CRAG (Yan et al., 2024)

Retrieval check	Separate fine-tuned evaluator (T5)
Correction	Open-ended, three confidence actions
Knowledge source	Adds large-scale web search
Deployment	Cloud / server, large models

This thesis

Prompted - same local model

At most one retry (bounded)

Closed local corpus, no web

Fully local, on a laptop

We keep the corrective idea, strip the machinery, and ask whether it still pays off locally.

X web search

X trained evaluator

X open-ended loop

X cloud scale

Methodology



1) Knowledge Ingestion Flow

1. Source Documents



PDFs



Web Pages



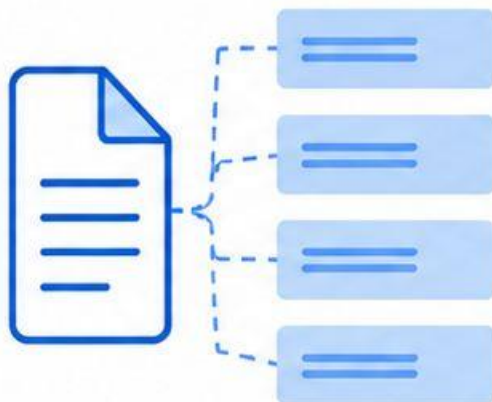
Databases



Notes



2. Chunking



Split documents into
smaller chunks



3. Embeddings

[0.12, -0.85, 0.33, ...]

[0.71, 0.21, -0.14, ...]

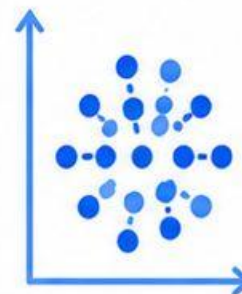
⋮

[-0.35, 0.62, 0.11, ...]

Convert chunks into
vector representations



4. Vector Database / Knowledge Base



Store and index vectors
for efficient similarity
search

Source: Image generated by OpenAI's DALL-E 3 via ChatGPT



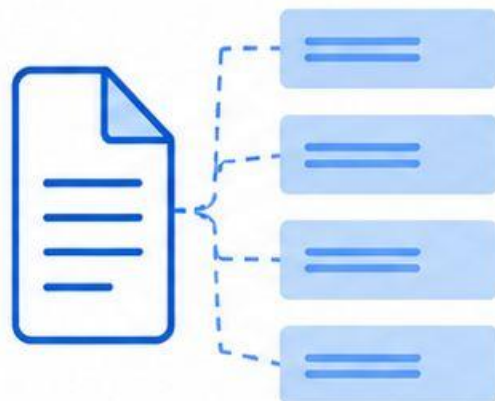
1) Knowledge Ingestion Flow

Closed corpus

- legal contracts & policies (LegalBench-mini)
- selected Wikipedia pages
- No web pages or external sources



2. Chunking



Split documents into smaller chunks



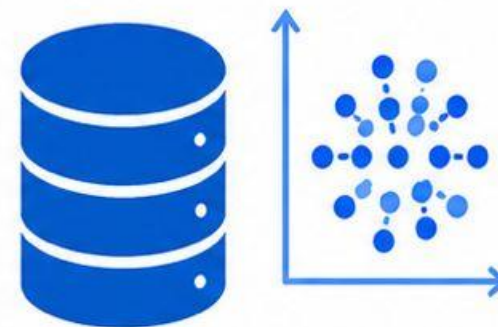
3. Embeddings

[0.12, -0.85, 0.33, ...]
[0.71, 0.21, -0.14, ...]
⋮
[-0.35, 0.62, 0.11, ...]

Convert chunks into vector representations



4. Vector Database / Knowledge Base



Store and index vectors for efficient similarity search

Source: Image generated by OpenAI's DALL-E 3 via ChatGPT



1) Knowledge Ingestion Flow

1. Source Documents



PDFs



Web Pages



Databases



Notes



Boundary-aware splitting

Documents are cut into **800-character** segments with **150-character overlap** to preserve clauses and sentences.



3. Embeddings

[0.12, -0.85, 0.33, ...]

[0.71, 0.21, -0.14, ...]

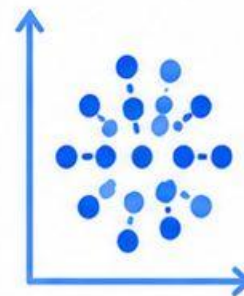
⋮

[-0.35, 0.62, 0.11, ...]

Convert chunks into
vector representations



4. Vector Database / Knowledge Base



Store and index vectors
for efficient similarity
search

Source: Image generated by OpenAI's DALL-E 3 via ChatGPT



1) Knowledge Ingestion Flow

1. Source Documents



PDFs



Web Pages

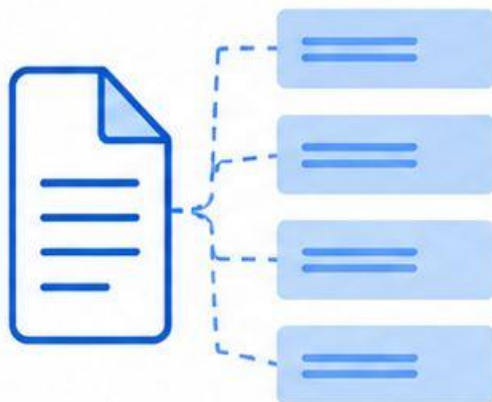


Databases



Notes

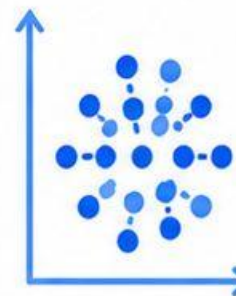
2. Chunking



Split documents into
smaller chunks

We encode each chunk
using the
BGE-base-en-v1.5
embedding model

4. Vector Database / Knowledge Base



Store and index vectors
for efficient similarity
search

Source: Image generated by OpenAI's DALL-E 3 via ChatGPT



1) Knowledge Ingestion Flow

1. Source Documents



PDFs



Web Pages



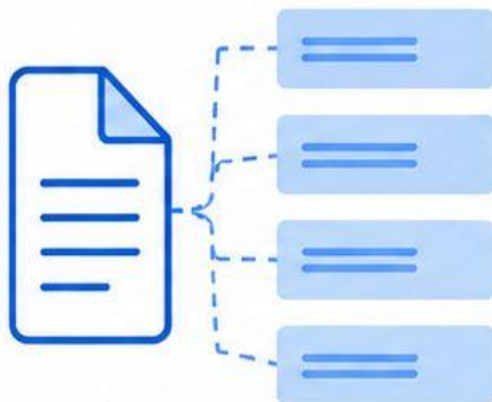
Databases



Notes



2. Chunking



Split documents into
smaller chunks



3. Embeddings

[0.12, -0.85, 0.33, ...]

[0.71, 0.21, -0.14, ...]

⋮

[-0.35, 0.62, 0.11, ...]

Convert chunks into
vector representations

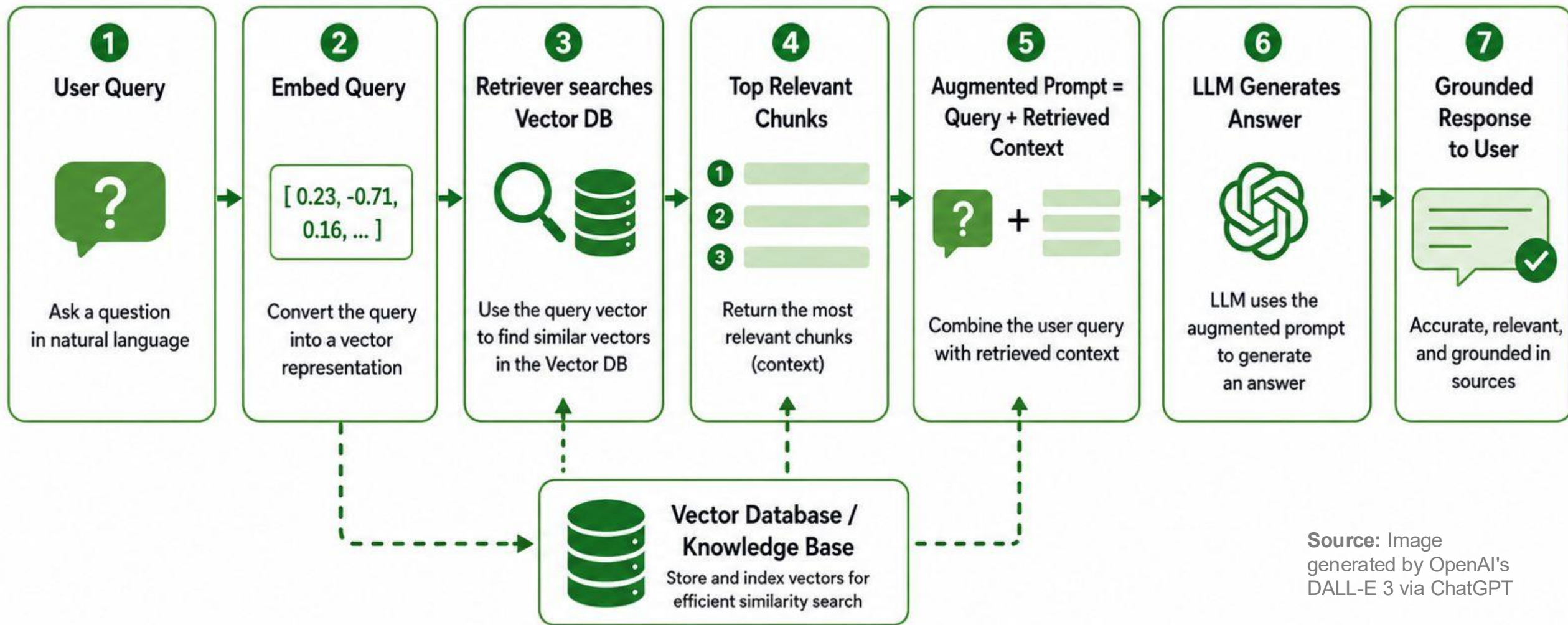


We store all
embeddings locally in a
FAISS index for fast
nearest-neighbour
search.

No remote database
needed.

Source: Image generated by OpenAI's DALL-E 3 via
ChatGPT

2) Question-Answering Flow



Source: Image generated by OpenAI's DALL-E 3 via ChatGPT



RAG improves LLM answers by retrieving relevant external knowledge before generation.

Architecture: shared retrieval, two pipelines

Shared retrieval backbone

chunk 800 / 150 → BGE-base embed → FAISS → top-5

Both pipelines consume the same top-5 context, then diverge:

Traditional

Generate

uses the top-5 directly — no correction

Bounded corrective

Controller
sufficient?



Rewrite +
re-retrieve k=20



RRF
fusion



Generate

Only the corrective path differs — at most one bounded retry, and the controller is the same local model that also generates.

Two benchmarks - two kinds of task

LegalBench-mini (GuhaEtAl2023LegalBench)

776 cases · 4 subsets

PrivacyQA · ContractNLI · MAUD · CUAD

Find the right clause in the right document

Evidence recall only - no gold answers

retrieval only

HotpotQA (YangEtAl2018HotpotQA)

1,000-case slice

Multi-hop: bridge + comparison

Combine facts across documents

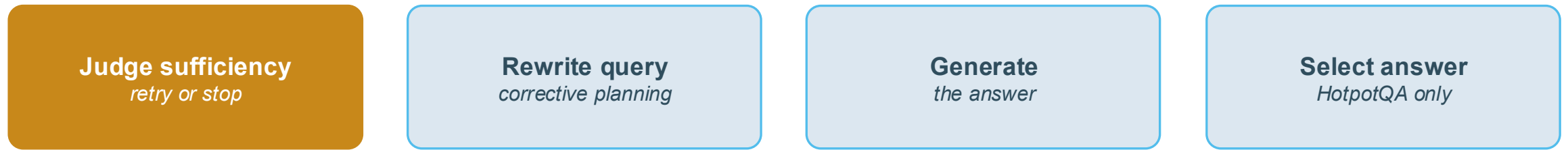
Supporting-title recall + EM / F1

retrieval + answer

Both are graded on retrieval; only HotpotQA adds answer scoring. Different structure → different demands on correction (RQ2).

The controller - one prompted model, four jobs

The corrective stage is driven by a single local model - **prompted, not trained** (no separate evaluator).



Two sufficiency checks: legal → source-focused (one document) · HotpotQA → global multi-document

Bounded to one retry - and because one model plays every role, it is also the main bottleneck.

Models and hardware - everything runs locally

Model	Family	Params	Quant.	Ctx.
<i>Laptop-local main models</i>				
Llama 3.2 3B Instruct	Llama 3.2	3B	Q4_K_M	4096
Llama 3.1 8B Instruct	Llama 3.1	8B	Q4_K_M	4096
Qwen 2.5 3B Instruct	Qwen 2.5	3B	Q4_K_M	4096
Qwen 2.5 7B Instruct	Qwen 2.5	7B	Q4_K_M	4096
<i>Cloud-scale reference models</i>				
Llama 3.1 70B Instruct	Llama 3.1	70B	Q4_K_M	4096
Qwen 2.5 72B Instruct	Qwen 2.5	72B	Q4_K_M	4096

Hardware: MacBook Air M4 · 16 GB unified memory · llama.cpp

What we measure?

Retrieval quality

Evidence / supporting-
title recall

Does correction find better evidence?

RQ1 · RQ2

Answer quality

Exact match, F1
(HotpotQA)

Does it produce better answers?

RQ1 · RQ2

Cost

Retry rate, latency,
peak memory

At what local cost?

RQ1 · RQ3

Measuring cost fairly, and validating results

Controlled latency study

Cost is measured separately from the accuracy runs.

Continuous runs drift with thermal and batch effects - so latency is isolated.

- fixed 100-query workloads, two reversed sweeps
- warm-ups + 300 s cooldowns, idle laptop
- median & 95th-pct latency, peak memory

Cross-environment validation

Effectiveness re-run on separate GPU hardware.

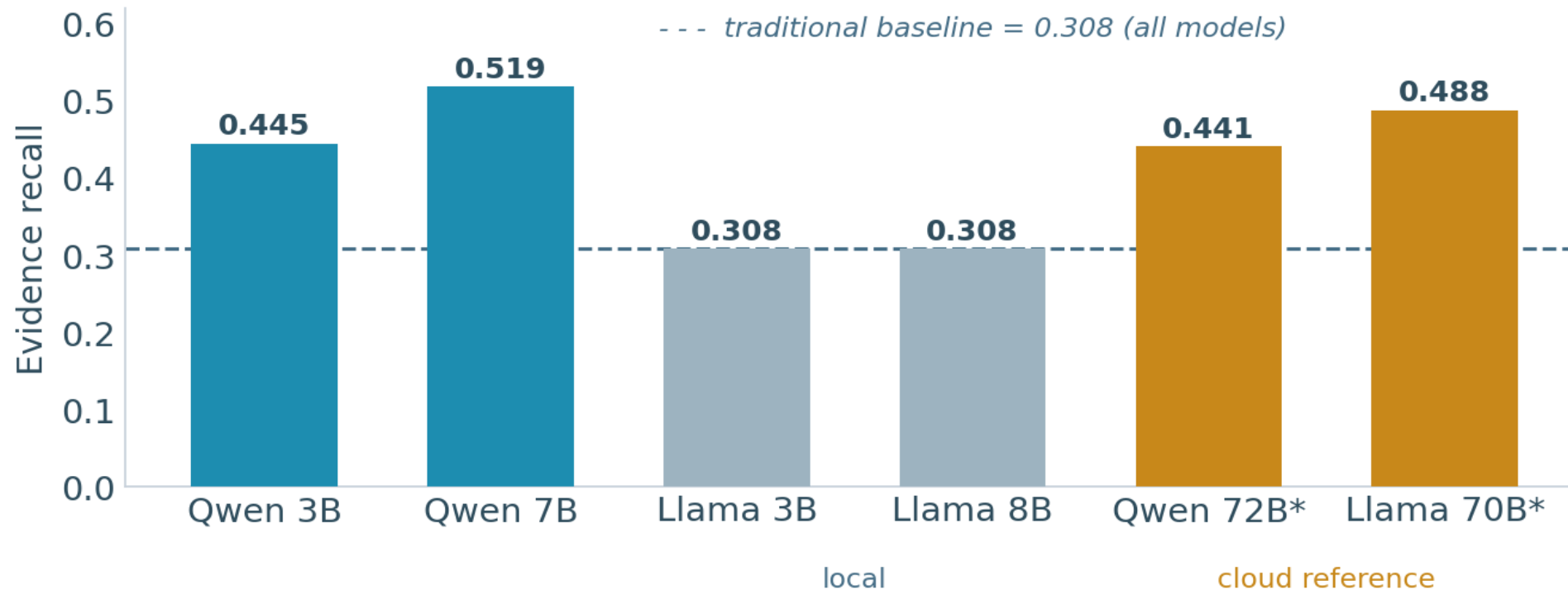
LLM outputs can shift across hardware - so we check the trends hold elsewhere.

- matched runs on a cloud RTX 4090 pod
- main effectiveness trends held
- latency / memory excluded (hardware-bound)

Cost reflects system behaviour, not noise - and the effectiveness trends reproduce across machines.

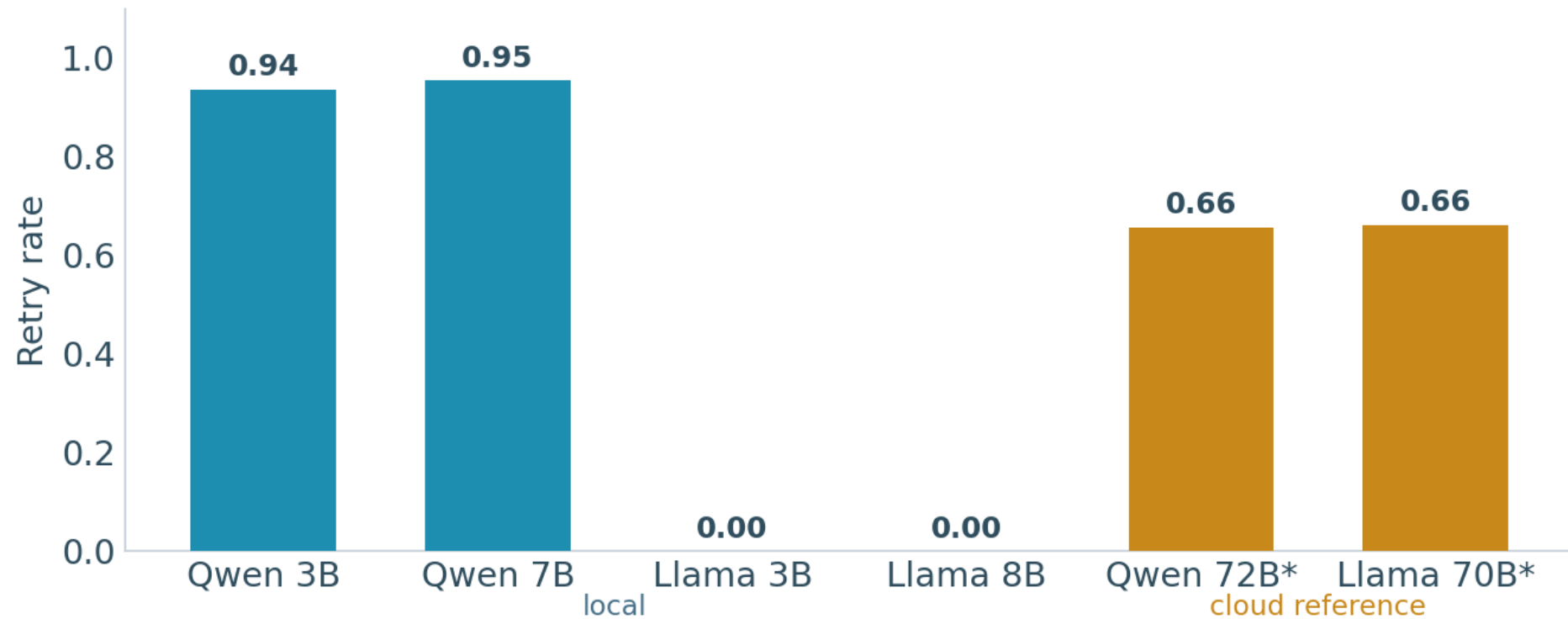
Results and discussion

LegalBench-mini: Evidence Recall

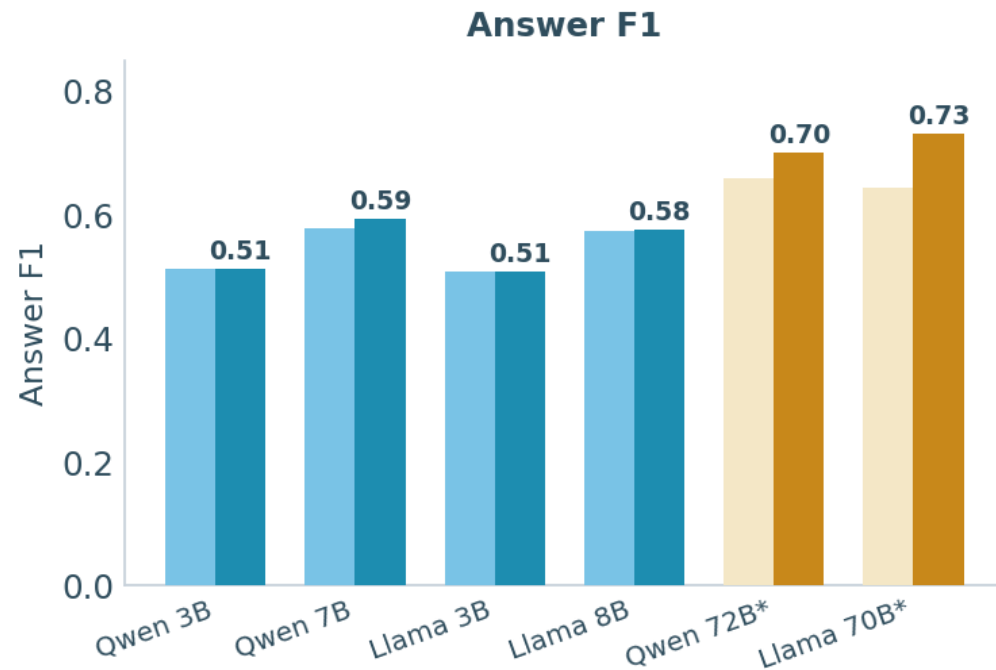
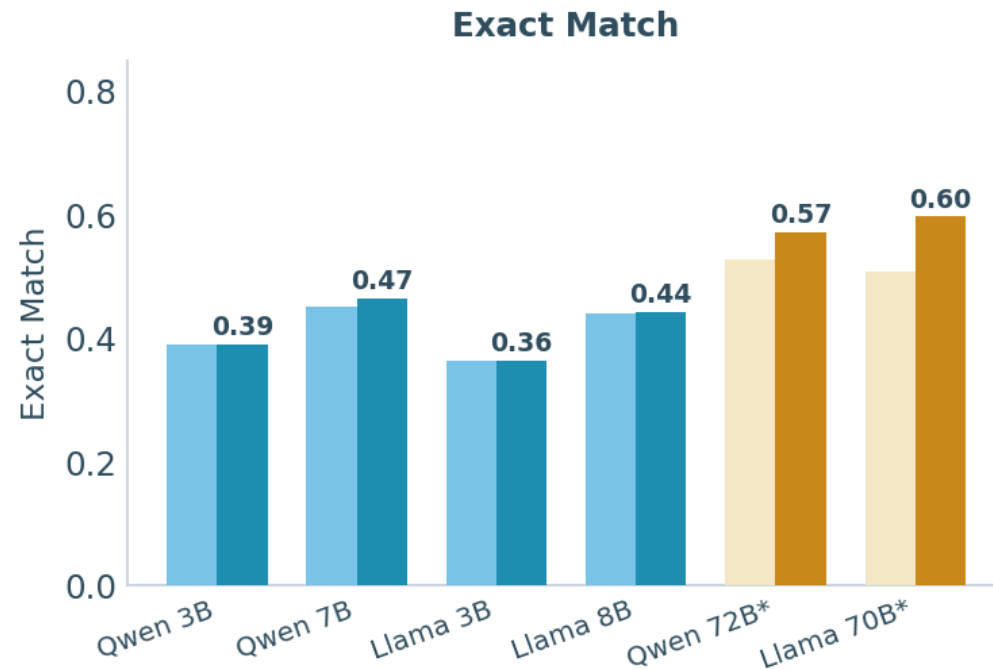


1. Traditional baseline recovers evidence in 239 / 776 cases
2. Qwen 7B corrective mode brings that up to 403 / 776 cases
3. Qwen 7b evidence recall is higher than LLM deployed on cloud
4. Llama models did not activate correction on LegalBench

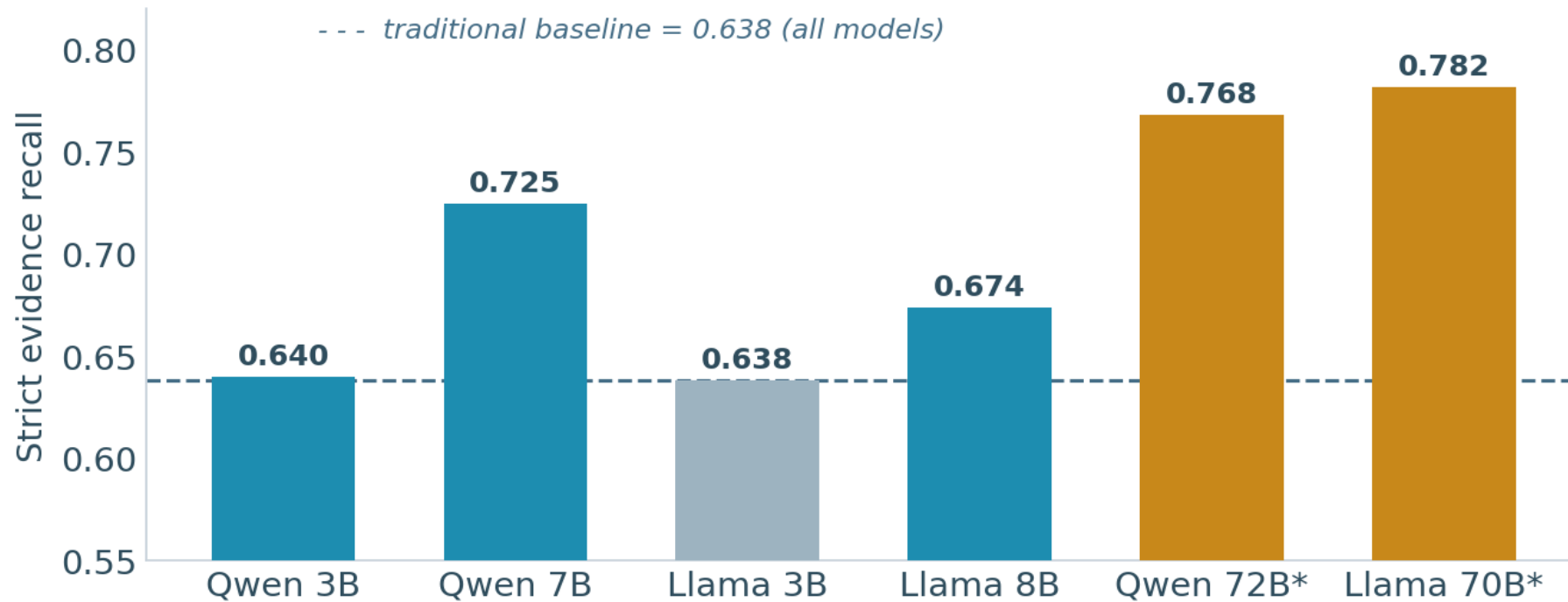
LegalBench-mini: Retry Rate



HotpotQA : Answer Correctness

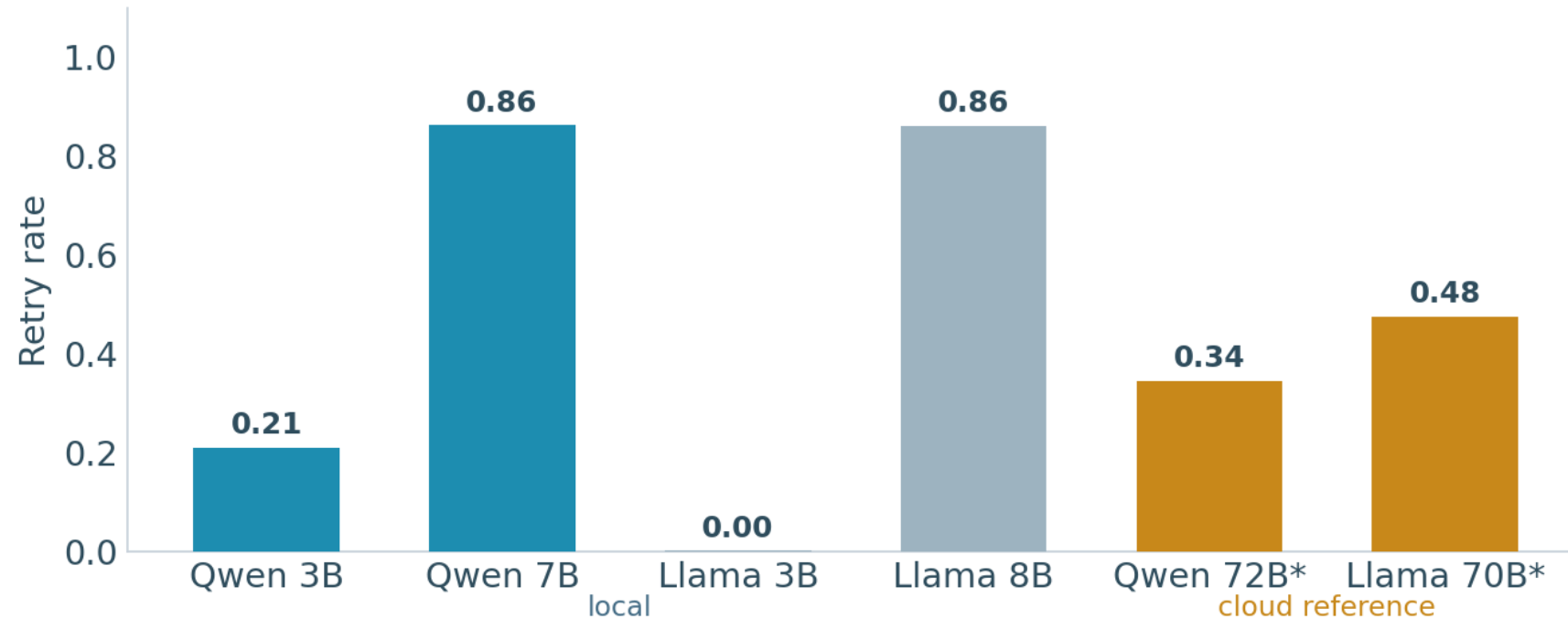


HotpotQA : Evidence recall



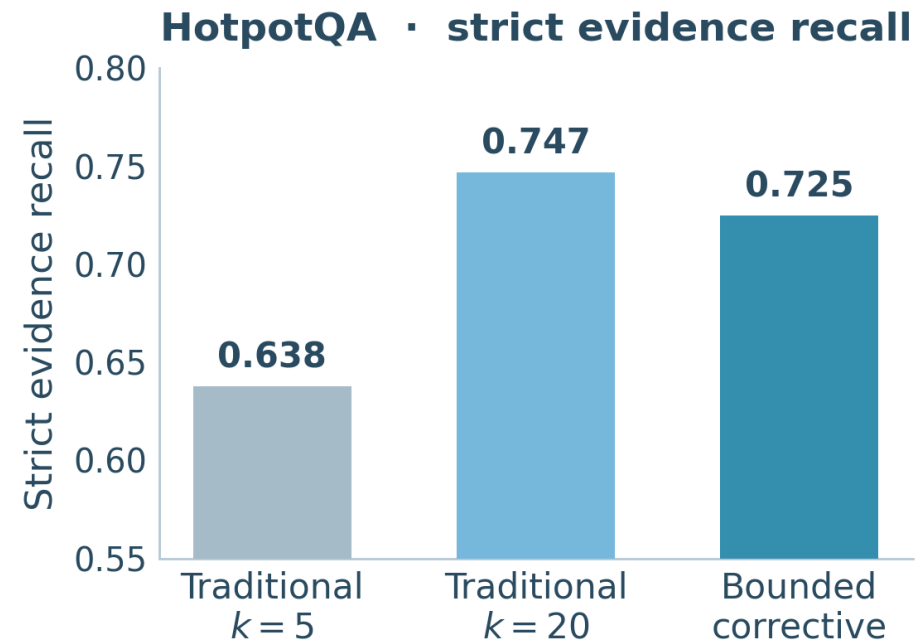
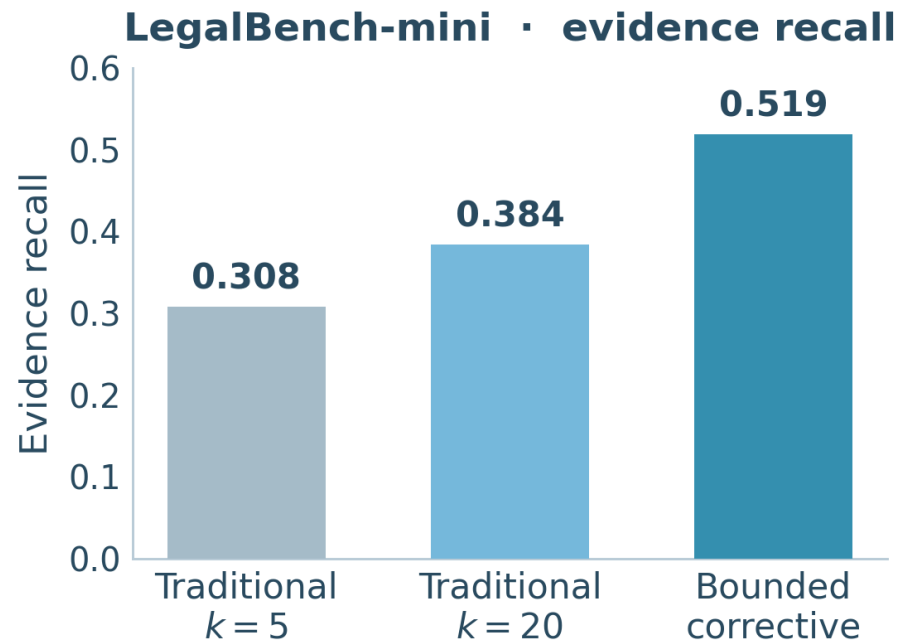
The first pass recovers both supporting evidences in 638 of 1,000 cases and exactly one evidence in another 342 cases

HotpotQA: Retry Rate



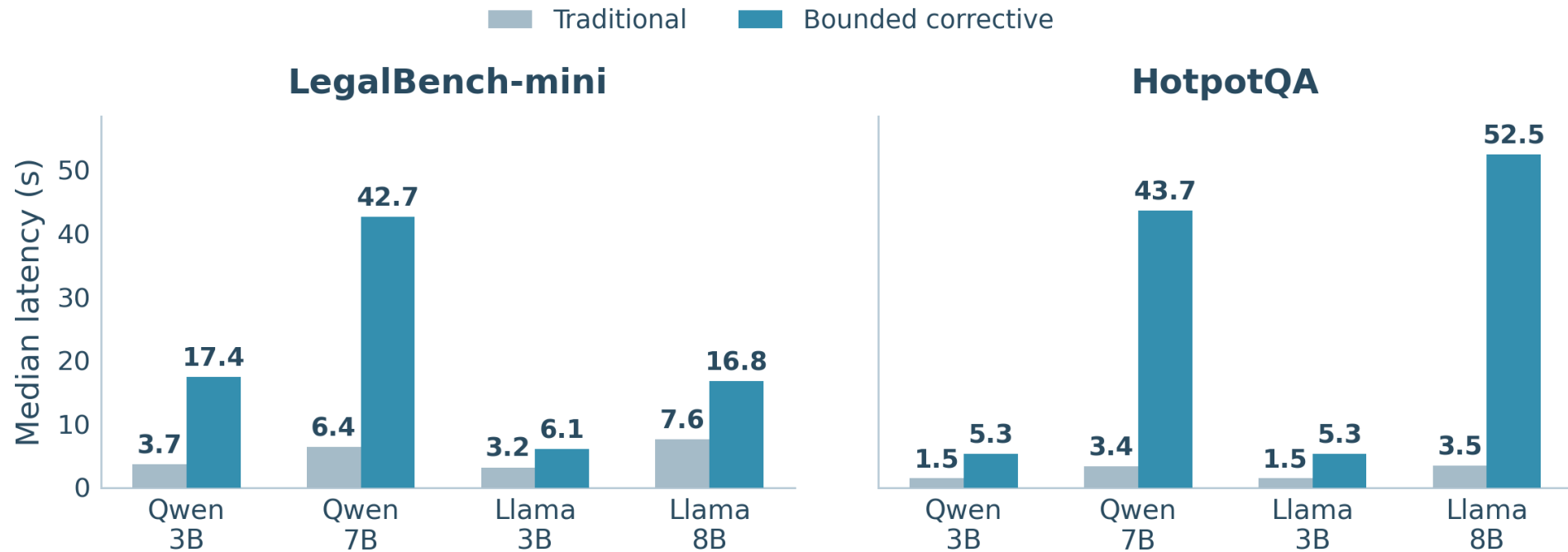
- Llama 8B retry
- Cloud scale models still retry more moderate

Qwen 7B diagnostic: correction or wider retrieval



- On **legalbench**, the gain is not explained by retrieval width alone
- **HotpotQA**: traditional k=20 matches or exceeds bounded correction.
- Wider one-pass retrieval is simpler and cheaper in this closed-corpus setup.

Latency



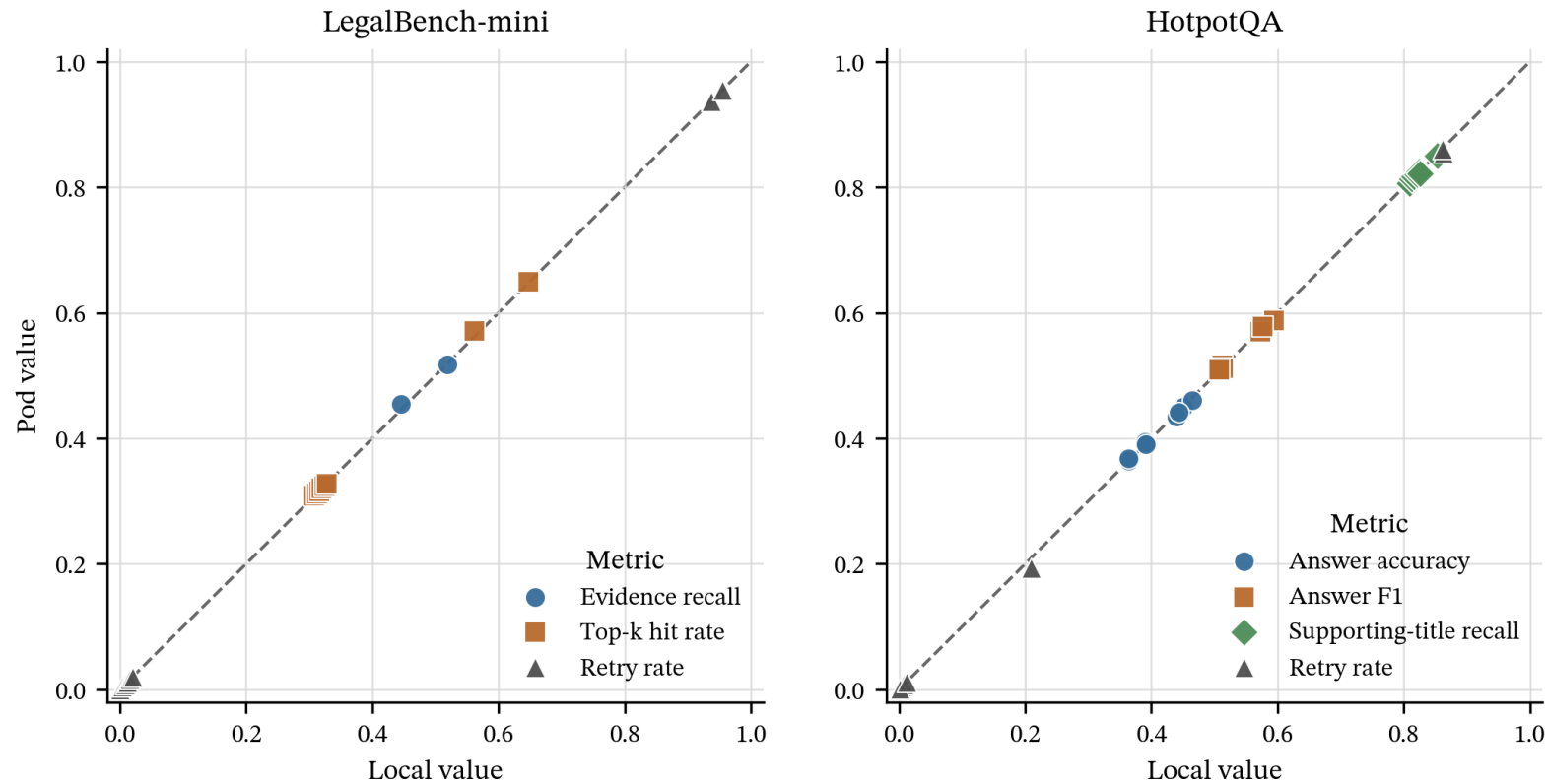
- Latency depends strongly on corrective-stage activation.
- High-activation runs exceed 40 s median latency.
- Qwen~3B offers the best local legal trade-off.
- HotpotQA corrective latency is difficult to justify when wider one-pass retrieval is competitive

Estimated Energy Cost per Query

Condition	Trad → Corrective	Multiplier
Qwen 7B, LegalBench	92 → 397 J	~4.3×
Qwen 7B, HotpotQA	57 → 363 J	~6.3×
Qwen 3B, LegalBench	55 → 216 J	~3.9×
Qwen 3B, HotpotQA	26 → 120 J	~4.6×

- Power estimations using *Apple powermetrics*
- Correction increases energy use
- Highest difference in memory usage is 0.33 GiB
- Heaviest load on the memory 6.32 GiB

The trends hold on other hardware



Exact agreement on effectiveness metrics on RTX 4090

Limitations

- **Pipeline-level evaluation**

We compare the two pipelines as whole systems

- **Within-study scope**

No full benchmarks

- **Fixed retrieval backbone**

One embedding model, one FAISS setup, fixed chunking not exhaustively tuned
some retrieval headroom is left unexplored.

Future work

- **Study the controller on its own**

compare the prompted same-model controller to a dedicated or trained sufficiency evaluator / router - measuring both quality and local runtime cost.

- **Compare against stronger one-pass retrieval**

hybrid retrieval, reranking, alternative embeddings, chunking and context-budget choices before crediting the correction loop.

- **Local RAG assistant**

The current prototype is preliminary and still needs testing before deployment.

Conclusions

- **Correction is retrieval repair.** It helped when the first pass missed and can hurt when it was already right.
- **Scale helped answer correction, not evidence recovery**
- **Tuning and calibrating the one-pass RAG is probably better on this laptop and setup.** On HotpotQA, k=20 retrieval is as good and far cheaper than correcting.
- **The controller is the bottleneck.** Activation varies by model and task; the same local model judges, rewrites, generates.
- **The cost is generation latency and energy not memory.** Worth it only when the quality gain justifies the extra time.

When correction helps: recovered evidence

Wordscapes · PrivacyQA

Query: Wordscapes policy - does it read my contacts?

First pass: gold missing; top chunk from 23andMe.txt

Controller: RETRY

Route: narrow to Wordscapes.txt

Final pass: gold recovered - top chunk Wordscapes.txt

✓ wrong document, then corrected

CEII / NDA · ContractNLI

Query: CEII NDA - bar disclosing it was negotiated?

First pass: gold missing; right doc (ceii-and-nda.txt)

Controller: RETRY

Route: re-search within ceii-and-nda.txt

Final pass: gold recovered - same document

✓ missed clause, then recovered

When the first pass misses, the bounded retry can recover the right evidence.

When correction hurts: lost evidence

Motorola · ContractNLI

Query: Motorola NDA - grants the party no rights?

First pass: gold already present (nda_form_motorola.txt)

Controller: RETRY - fired anyway

Route: rights-focused rewrite

Final pass: gold lost - shifted to NCDG NDA.txt

✗ good pass broken — clause drift

HNBA · ContractNLI

Query: HNBA - notify before a required disclosure?

First pass: gold already present (HNBA agreement)

Controller: RETRY - fired anyway

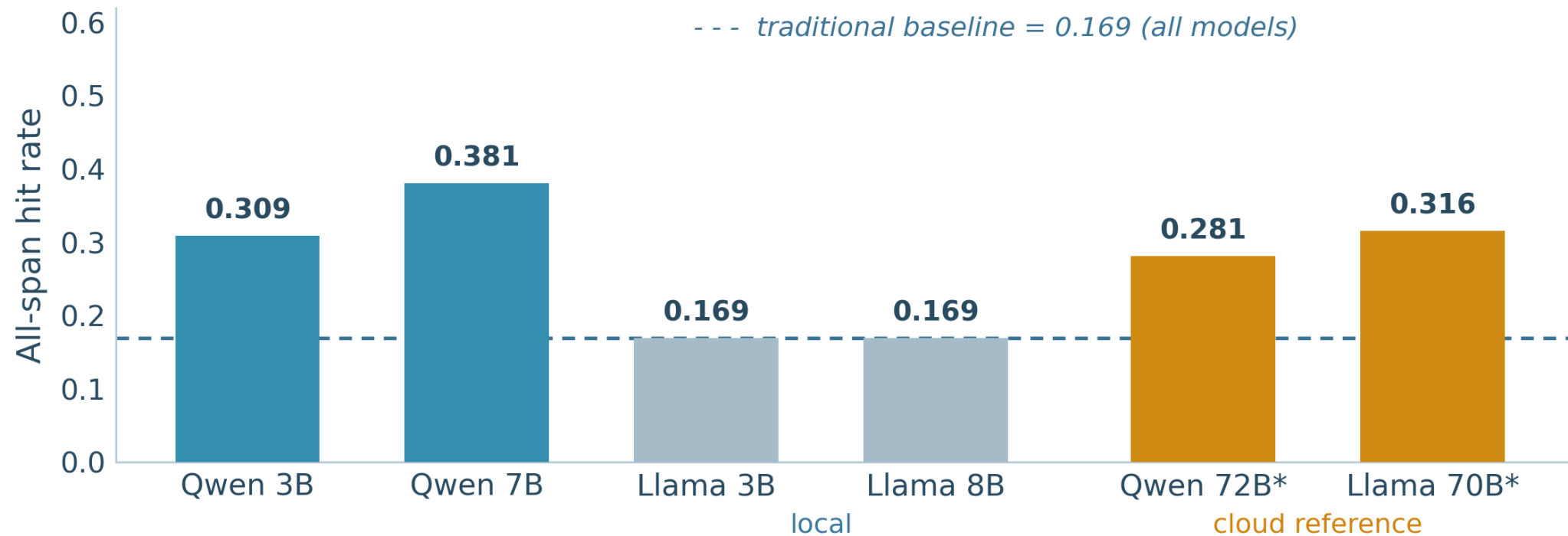
Route: narrowed to ENERGOUS alliance agreement

Final pass: gold lost - shifted to IPTK mutual NDA

✗ good pass broken — wrong-doc routing

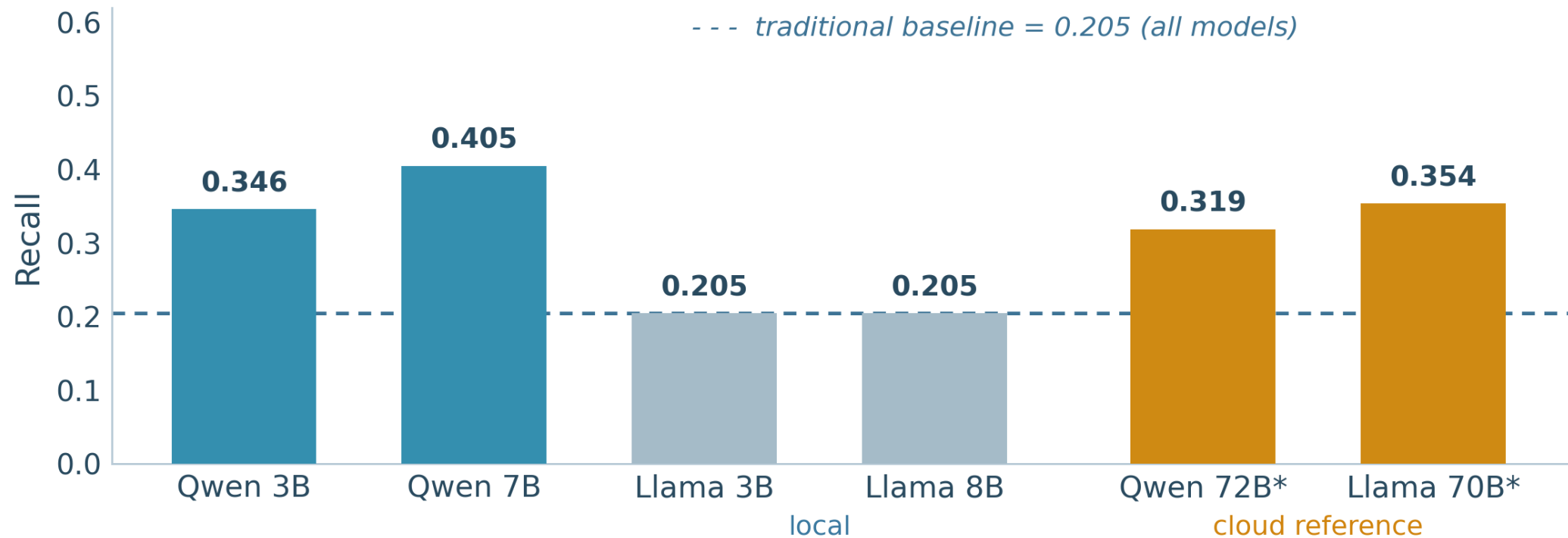
When the first pass was already right, the retry can overwrite good evidence — this is the cost.

LegalBench-mini: stricter all-span evaluation



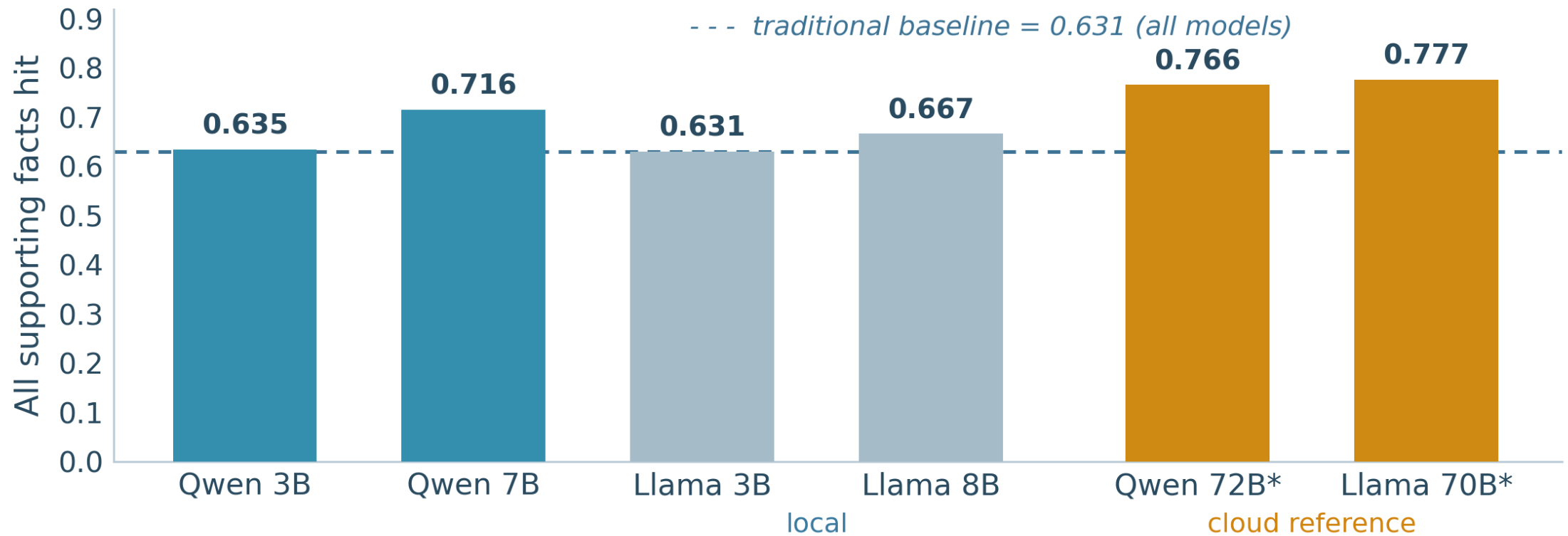
A case counts as successful only when the final context overlaps every annotated span.

LegalBench-mini: recall



Recall measures the proportion of annotated evidence characters covered by the final context.

HotpotQA: sentence level evaluation



The sentence-level values are slightly lower than the reported title-level values, but the largest difference is nine cases out of one thousand.

HotpotQA : cloud-scale models retries more selectively

Model	Deployment	Total retries	Retry: missing support	Retry: support already complete	Net recovered cases
Qwen 7B	local	863	340	523	+87
Llama 8B	local	862	341	521	+36
Qwen 72B	cloud reference	345	233	112	+130
Llama 70B	cloud reference	476	278	198	+144

Shared first pass: 362 queries miss at least one supporting page; 638 already contain both.
Pattern: *cloud-scale models retry less often but recover more missing evidence.*

Legalbench prompt

Question:
{question}

Retrieved Context:
{context}

Decide whether the retrieved context is sufficient to answer the question. Return STOP if the retrieval is relevant and already contains enough evidence to answer the question. If the likely answer span or clause is already present, return STOP even if another search might find a cleaner snippet. Otherwise return RETRY

HotpotQA prompt

Original Question:

{question}

Retrieved Context:

{context}

Decide whether another global retrieval is needed. If not needed, set need_retry to NO and set both queries to NONE. If needed, set need_retry to YES and write a step-back query for the missing relation or bridge concept. Use backup_query only when it adds a complementary concrete follow-up, not just a rephrasing.

Return JSON with this schema:

```
{  
  "need_retry": "<YES or NO>",  
  "missing_fact": "<short phrase describing the missing evidence>",  
  "primary_query": "<short keyword step-back query or NONE>",  
  "backup_query": "<optional short complementary follow-up query or NONE>" }
```